Intro to R - Data Wrangling Basics - Example Solution

B.A. Hartlaub and A.S. Wagaman

Data Wrangling

One of the most basic aspects of data analysis is working with the data. This includes getting the data into the form that you want it for analysis - picking a subset of the observations, adding new variables, working with only a few variables.

We want to introduce you to (or review with you) a series of data *verbs* (R functions) that will be useful for data *wrangling*.

The data set is from a paper called "Using Data Mining To Predict Secondary School Student Alcohol Consumption" by Fabio Pagnotta and Hossain Mohammad Amran of the Department of Computer Science, University of Camerino, and the data set is hosted online in UCI's machine learning repository.

The data set contains a number of variables about students and their performance in courses, as well as alcohol consumption. Just skim this over. The idea is that we needed a somewhat large, more complicated than just simple linear regression, data set to work with.

(Copied from provided codebook online) Attributes for both student-mat.csv (Math course) and student-por.csv (Portuguese language course) datasets:
1 school - student's school (binary: 'GP' - Gabriel Pereira or 'MS' - Mousinho da Silveira)
2 sex - student's sex (binary: 'F' - female or 'M' - male)
3 age - student's age (numeric: from 15 to 22)
4 address - student's home address type (binary: 'U' - urban or 'R' - rural)
5 famsize - family size (binary: 'LE3' - less or equal to 3 or 'GT3' - greater than 3)
6 Pstatus - parent's cohabitation status (binary: 'T' - living together or 'A' - apart)
7 Medu - mother's education (numeric: 0 - none, 1 - primary education (4th grade), 2 - 5th to 9th grade, 3 - secondary education or 4 - higher education)
8 Fedu - father's education (numeric: 0 - none, 1 - primary education (4th grade), 2 - 5th to 9th grade, 3 - secondary education or 4 - higher education)
9 Mjob - mother's job (nominal: 'teacher', 'health' care related, civil 'services' (e.g. administrative or police), 'at_home' or 'other')
10 Fjob - father's job (nominal: 'teacher', 'health' care related, civil 'services' (e.g. administrative or police), 'at_home' or 'other')
11 reason - reason to choose this school (nominal: close to 'home', school 'reputation', 'course' preference or 'other')
12 guardian - student's guardian (nominal: 'mother', 'father' or 'other')
13 traveltime - home to school travel time (numeric: 1 - <15 min., 2 - 15 to 30 min., 3 - 30 min. to 1 hour, or 4 - >1 hour)
14 studytime - weekly study time (numeric: 1 - <2 hours, 2 - 2 to 5 hours, 3 - 5 to 10 hours, or 4 - >10 hours)
15 failures - number of past class failures (numeric: n if 1<=n<3, else 4)
16 schoolsup - extra educational support (binary: yes or no)
17 famsup - family educational support (binary: yes or no)
18 paid - extra paid classes within the course subject (Math or Portuguese) (binary: yes or no)
19 activities - extra-curricular activities (binary: yes or no)
20 nursery - attended nursery school (binary: yes or no)
21 higher - wants to take higher education (binary: yes or no)

22 internet - Internet access at home (binary: yes or no)
23 romantic - with a romantic relationship (binary: yes or no)
24 famrel - quality of family relationships (numeric: from 1 - very bad to 5 - excellent)
25 freetime - free time after school (numeric: from 1 - very low to 5 - very high)
26 goout - going out with friends (numeric: from 1 - very low to 5 - very high)
27 Dalc - workday alcohol consumption (numeric: from 1 - very low to 5 - very high)
28 Walc - weekend alcohol consumption (numeric: from 1 - very low to 5 - very high)
29 health - current health status (numeric: from 1 - very bad to 5 - very good)
30 absences - number of school absences (numeric: from 0 to 93)

Finally, the grades are related with the course subject, Math or Portuguese:
31 G1 - first period grade (numeric: from 0 to 20)
31 G2 - second period grade (numeric: from 0 to 20)
32 G3 - final grade (numeric: from 0 to 20, output target)

Working with Data

The data was provided as two different .csv files online. We obtained some errors trying to work with them, so we ended up saving them as .txt files on the web.

The idea here is that there are values for the same students in both the -mat and -por files. So we need to get the data sets into R and then merge (join) them so we don't count students twice, but we still get a large amount of data.

To load the data, you need to run the following commands:

```
d1 <- read.table("https://awagaman.people.amherst.edu/stat230/student-mat.txt",
                 sep = ";",header=TRUE)
d2 <- read.table("https://awagaman.people.amherst.edu/stat230/student-por.txt",
                 sep = ";",header=TRUE)

studydata <- merge(d1, d2, by = c("school", "sex", "age", "address", "famsize",
                                  "Pstatus", "Medu", "Fedu", "Mjob", "Fjob",
                                  "reason", "nursery", "internet"))
print(nrow(studydata)) # 382 students
```

```
## [1] 382
```

What does this code do? It loads both data sets - for math and Portuguese courses. Then, it looks for the same students in each, and merges the data sets based on the variables provided in the BY statement. The overlap is 382 students. Merging files is common, and just one aspect of data wrangling. A similar operation can be accomplished with join, but we used merge here because that was what was supplied by the authors as part of their workflow. Both functions may be useful for your later work in R.

Now that we have the data, let's look at the commands that we wanted to introduce you to. These are examples, and there are MANY more functions that could be useful to you, but hopefully this is a good start. All these functions "live" in the dplyr package. Dplyr is loaded with the mosaic package (which you should have loaded at the top).

Data Verbs

Each of the functions below is a data verb - it takes an input of a data set and does something with it - outputting either another data set or a summary statistic, for example.

Rename

We didn't name the variables in the data set ourselves. When we merged the files, variables in common to both data sets were used to perform the merge. Any variable only in the first data set (math scores) got a .x appended to its name, and any variable only in the second data set (Portuguese) got a .y appended to its name. You can see this with:

```
names(studydata)
```

```
##  [1] "school"       "sex"          "age"          "address"      "famsize"
##  [6] "Pstatus"      "Medu"         "Fedu"         "Mjob"         "Fjob"
## [11] "reason"       "nursery"      "internet"     "guardian.x"   "traveltime.x"
## [16] "studytime.x"  "failures.x"   "schoolsup.x"  "famsup.x"     "paid.x"
## [21] "activities.x" "higher.x"     "romantic.x"   "famrel.x"     "freetime.x"
## [26] "goout.x"      "Dalc.x"       "Walc.x"       "health.x"     "absences.x"
## [31] "G1.x"         "G2.x"         "G3.x"         "guardian.y"   "traveltime.y"
## [36] "studytime.y"  "failures.y"   "schoolsup.y"  "famsup.y"     "paid.y"
## [41] "activities.y" "higher.y"     "romantic.y"   "famrel.y"     "freetime.y"
## [46] "goout.y"      "Dalc.y"       "Walc.y"       "health.y"     "absences.y"
## [51] "G1.y"         "G2.y"         "G3.y"
```

Note that this means some of the variables are appearing TWICE now (not really optimal), but we can keep track of which data set they came from. However, we might want to change some variable names to make things clearer than "x" and "y".

For example, G1.x is the first period math grade. We can rename it using:

```
studydata <- rename(studydata, FirstMathGrade = G1.x)
```

Note the form here is: dataset <- function(dataset, newvariablename = oldvariablename). It WILL overwrite the original data set, because we used the same dataset name in each place. Be careful with overwriting (though here, that's exactly what we want).

Let's rename the rest of the math grades- you can string these together in a single call to the function:

```
studydata <- rename(studydata, SecondMathGrade = G2.x, ThirdMathGrade = G3.x)
names(studydata) #check that it worked
```

```
##  [1] "school"          "sex"             "age"             "address"
##  [5] "famsize"         "Pstatus"         "Medu"            "Fedu"
##  [9] "Mjob"            "Fjob"            "reason"          "nursery"
## [13] "internet"        "guardian.x"      "traveltime.x"    "studytime.x"
## [17] "failures.x"      "schoolsup.x"     "famsup.x"        "paid.x"
## [21] "activities.x"    "higher.x"        "romantic.x"      "famrel.x"
## [25] "freetime.x"      "goout.x"         "Dalc.x"          "Walc.x"
## [29] "health.x"        "absences.x"      "FirstMathGrade"  "SecondMathGrade"
## [33] "ThirdMathGrade"  "guardian.y"      "traveltime.y"    "studytime.y"
## [37] "failures.y"      "schoolsup.y"     "famsup.y"        "paid.y"
## [41] "activities.y"    "higher.y"        "romantic.y"      "famrel.y"
## [45] "freetime.y"      "goout.y"         "Dalc.y"          "Walc.y"
## [49] "health.y"        "absences.y"      "G1.y"            "G2.y"
## [53] "G3.y"
```

Your turn. Rename the Portuguese grades to FirstPortGrade, etc.

```
studydata <- rename(studydata, FirstPortGrade = G1.y,
                    SecondPortGrade = G2.y,
                    ThirdPortGrade = G3.y)
names(studydata)
```

```
##  [1] "school"          "sex"             "age"             "address"
##  [5] "famsize"         "Pstatus"         "Medu"            "Fedu"
##  [9] "Mjob"            "Fjob"            "reason"          "nursery"
## [13] "internet"        "guardian.x"      "traveltime.x"    "studytime.x"
## [17] "failures.x"      "schoolsup.x"     "famsup.x"        "paid.x"
## [21] "activities.x"    "higher.x"        "romantic.x"      "famrel.x"
## [25] "freetime.x"      "goout.x"         "Dalc.x"          "Walc.x"
## [29] "health.x"        "absences.x"      "FirstMathGrade"  "SecondMathGrade"
## [33] "ThirdMathGrade"  "guardian.y"      "traveltime.y"    "studytime.y"
## [37] "failures.y"      "schoolsup.y"     "famsup.y"        "paid.y"
## [41] "activities.y"    "higher.y"        "romantic.y"      "famrel.y"
## [45] "freetime.y"      "goout.y"         "Dalc.y"          "Walc.y"
## [49] "health.y"        "absences.y"      "FirstPortGrade"  "SecondPortGrade"
## [53] "ThirdPortGrade"
```

Filter

What if we only want to select certain observations out of the data set? We can easily do that, as long as we know what our selection criteria are.

The address variable is: address - student's home address type (binary: 'U' - urban or 'R' - rural)

So, if we want to just look at the rural students, we could do:

```
ruralstudydata <- filter(studydata, address == "R")
```

What does this do? It creates a NEW data set called ruralstudy data, using studydata, and only pulling out students whose address variable was "R".

The form of the function is: newdataset <- filter(dataset, variable expression criteria)

The double equals sign means that a categorical variable must be EQUAL to that value. For numeric values, you can use <, >, etc. You can also chain these together with "and" or "or" expressions. Again, just trying to introduce you to the idea that these manipulations are possible.

Here are some other examples:

```
HighAlc <- filter(studydata, Dalc.y > 4) #high weekday alcohol values from Port data set
Mjobhealthteacher <- filter(studydata, Mjob == "health" | Mjob == "teacher")
#mother's job is in health OR teacher
```

This second example uses an "OR" (the bar) (it adds a complication that Mjob still thinks it has 5 levels but only has 2 in this data set, but we'll deal with such issues later). You can also string conditions together for multiple variables:

```
combineddata <- filter(studydata,
                       Mjob == "other",
                       FirstMathGrade > 10,
                       address == "R")
```

How many students are in the combineddata data set? What conditions do these students satisfy?

```
dim(combineddata)
```

```
## [1] 11 53
```

There are 11 students in this data set. You could see this in the workspace too. Based on the command, their mother's job was recorded as "Other", they have a FirstMathGrade higher than 10, and they have a rural address.

Create an urban student data set for students with a SecondMathGrade larger than 10, starting from the studydata dataset.

```
urbanSecondHigh <- filter(studydata, address == "U", SecondMathGrade > 10)
```

You should end up with 160 observations (students) in this data set.

Notice that rather than overwriting the original studydata here, we always chose a NEW data set name. This is so the original data are always available to us (if you accidentally overwrite, you just need to run the starting load commands again).

Select

Select lets us create data sets with only certain variables in them. So, filter lets you pick rows/observations, and select works the same way but for columns/variables.

```
studydatasmall <- dplyr::select(studydata, sex, age, address, FirstMathGrade,
                                SecondMathGrade, ThirdMathGrade, Dalc.x, Walc.x)
#can use select, but if it gives an error, force it to use dplyr's select
```

See if you can figure out how this function is structured.

```
summary(studydatasmall)
```

```
##      sex                 age          address          FirstMathGrade
##  Length:382         Min.   :15.00   Length:382         Min.   : 3.00
##  Class :character   1st Qu.:16.00   Class :character   1st Qu.: 8.00
##  Mode  :character   Median :17.00   Mode  :character   Median :10.50
##                     Mean   :16.59                      Mean   :10.86
##                     3rd Qu.:17.00                      3rd Qu.:13.00
##                     Max.   :22.00                      Max.   :19.00
##  SecondMathGrade ThirdMathGrade      Dalc.x          Walc.x
##  Min.   : 0.00   Min.   : 0.00   Min.   :1.000   Min.   :1.00
##  1st Qu.: 8.25   1st Qu.: 8.00   1st Qu.:1.000   1st Qu.:1.00
##  Median :11.00   Median :11.00   Median :1.000   Median :2.00
##  Mean   :10.71   Mean   :10.39   Mean   :1.474   Mean   :2.28
##  3rd Qu.:13.00   3rd Qu.:14.00   3rd Qu.:2.000   3rd Qu.:3.00
##  Max.   :19.00   Max.   :20.00   Max.   :5.000   Max.   :5.00
```

Mutate

What if we want to add a new variable to the data set? Mutate lets us do that. In the studydatasmall data set, we have weekend and weekday alcohol consumption. What if we wanted to "average" that? You might weight the weekday value by 5, and weekend value by 2 before dividing by 7 to get a realistic average, right?

```
studydatasmall <- mutate(studydatasmall, AvgAlc = (5*Dalc.x + 2*Walc.x)/7)
```

This adds a new variable called AvgAlc to the existing studydatasmall data set.

Piping

You may see code in places where a scheme called piping is used. The idea is to reduce some of the extra steps where we save code pieces. The output can be passed from one line to the next without saving it explicitly. Here's an example of code with piping and without, that also includes some extra data verbs that you may see or want to learn about: group_by and summarise.

```
studydatatemp <- filter(studydata, school == "GP", sex == "F")
studydatatemp <- mutate(studydatatemp, AvgAlc = (5*Dalc.x + 2*Walc.x)/7)
```

```
studydatatemp2 <- group_by(studydatatemp, famsize)
summarise(studydatatemp2, mean = mean(AvgAlc), n = n())
```

```
## # A tibble: 2 x 3
##   famsize  mean     n
##   <chr>   <dbl> <int>
## 1 GT3      1.43   135
## 2 LE3      1.56    40
```

This creates a new data set with only female students in GP schools, computes their average alc value (weighted as above), then outputs the mean of that variable for groups determined by the variable famsize. (You can probably figure out other ways of doing this too.)

Here is the piping version:

```
studydatatemp %>%
  filter(school == "GP", sex == "F") %>%
  mutate(AvgAlc = (5*Dalc.x + 2*Walc.x)/7) %>%
  group_by(famsize) %>%
  summarise(mean = mean(AvgAlc), n = n())
```

```
## # A tibble: 2 x 3
##   famsize  mean     n
##   <chr>   <dbl> <int>
## 1 GT3      1.43   135
## 2 LE3      1.56    40
```

```
# there's a newer pipe you could use too!
studydatatemp |>
  filter(school == "GP", sex == "F") |>
  mutate(AvgAlc = (5*Dalc.x+2*Walc.x)/7) |>
  group_by(famsize) |>
  summarise(mean = mean(AvgAlc), n = n())
```

```
## # A tibble: 2 x 3
##   famsize  mean     n
##   <chr>   <dbl> <int>
## 1 GT3      1.43   135
## 2 LE3      1.56    40
```

Practice

Start with the studydata data set.

absences.x and absences.y should be MathAbsences and PortAbsences, respectively. Fix this.

```
studydata <- rename(studydata,
                    MathAbsences = absences.x,
                    PortAbsences = absences.y)
```

We want to look at students who did NOT have perfect attendance in at least one class. Create a new data set for us to use.

```
# the BAR is for an "or" and & is used for "and" - there is more to learn here
notPA <- filter(studydata,
                MathAbsences > 0 | PortAbsences > 0)
```

Working with THOSE students, suppose we only care about the student's age, sex, Portuguese

class absences, weekday alcohol from the Portuguese class data (Dalc.y), and three Portuguese scores. Create a new data set for us to work from.

```
notPAsmall <- dplyr::select(notPA, age, sex, PortAbsences, Dalc.y,
                            FirstPortGrade, SecondPortGrade, ThirdPortGrade)
```

Add the average Portuguese grade to the data set.

NOTE: This grade may not be meaningful, but we wanted you to add some variable to the data set.

```
notPAsmall <- mutate(notPAsmall, AvgPortGrade =
                       (FirstPortGrade + SecondPortGrade + ThirdPortGrade)/3)
```

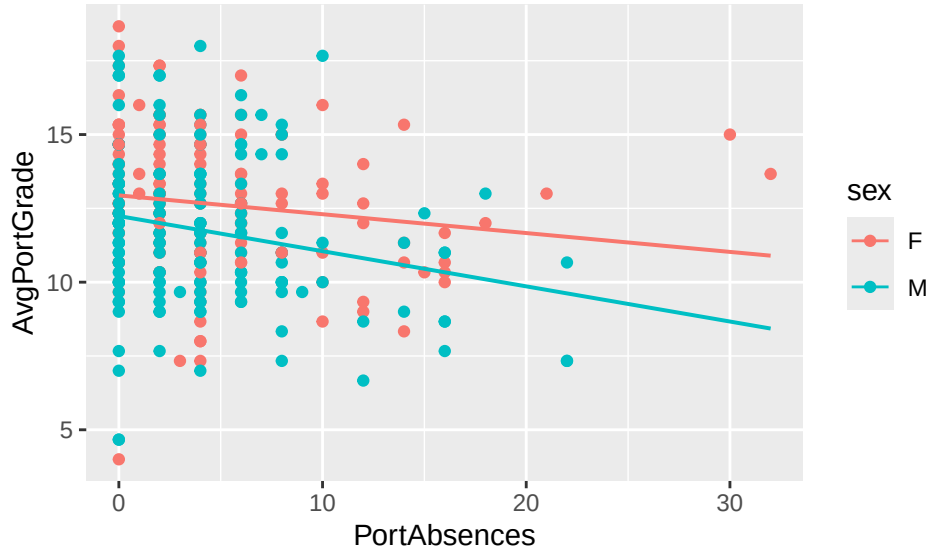
Using appropriate plots, examine some relationships of interest to you in this data set. For example, for this subset of students, are grades and absences related? How about age and weekday alcohol use? Absences and alcohol use?

```
names(notPAsmall)
```

```
## [1] "age"             "sex"             "PortAbsences"    "Dalc.y"
## [5] "FirstPortGrade"  "SecondPortGrade" "ThirdPortGrade"  "AvgPortGrade"
```

```
gf_point(AvgPortGrade ~ PortAbsences, color = ~ sex, data = notPAsmall) %>%
  gf_lm()
```

```
## Warning: Using the `size` aesthetic with geom_line was deprecated in ggplot2 3.4.0.
## i Please use the `linewidth` aesthetic instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Did you find anything interesting?

Was not expecting the negative correlation between absences and average grade that we see in the plots. There is surely more of interest here!

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken.

Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

Data

"Using Data Mining To Predict Secondary School Student Alcohol Consumption" by Fabio Pagnotta and Hossain Mohammad Amran of the Department of Computer Science, University of Camerino, and the data set is hosted online in UCI's machine learning repository.

License